Onboarding Accelerator — Microsoft Defender for Cloud Apps

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Onboarding Accelerator: Microsoft Defender for Cloud Apps 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 5

Effort . . . 6

1. Solution Overview — Onboarding Accelerator: Microsoft Defender for Cloud Apps 6

What Microsoft Defender for Cloud Apps is . . . 6

Why this matters . . . 6

How it works . . . 7

What you can control . . . 7

Limitations (be honest with buyers) . . . 7

Connections and authentication used . . . 8

The standard deployment model . . . 8

2. Requirements Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps 8

2.1 Defender for Cloud Apps activated . . . 8

2.2 Priority SaaS apps and connector credentials . . . 8

2.3 Cloud Discovery log source . . . 9

2.4 App governance enabled . . . 9

2.5 Entra Conditional Access for App Control . . . 9

2.6 Pilot user group defined . . . 9

2.7 Policy posture and enforcement governance . . . 9

Checklist . . . 10

Global Requirements — Defender (category-level) 10

G1. Microsoft 365 / Entra tenant . . . 10

G2. Defender licensing . . . 10

G3. Administrative roles (least privilege) . . . 10

G4. Onboarding reach to endpoints / identity / workloads . . . 11

G5. Network access to Defender service endpoints . . . 11

G6. A pilot ring . . . 11

G7. Log Analytics workspace (where required) . . . 11

G8. Response governance . . . 11

3. Implementation Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps 11

Naming (set once, reuse verbatim) . . . 12

Step 1 — Assess readiness . . . 12

Step 2 — Connect the priority apps (API connectors) . . . 12
Step 3 — Turn on Cloud Discovery . . . 12
Step 4 — Enable app governance and OAuth app policy (report-only) . . . 13
Step 5 — Create session/access policies via Conditional Access App Control (pilot, report-only) . . . 13
Step 6 — Add anomaly-detection and activity policies (monitor-first) . . . 13
Step 7 — Validate signal and control on the pilot . . . 13
Step 8 — Enforce and expand tenant-wide . . . 14
5. Testing Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps 14
Test 1 — App connectors connected . . . 14
Test 2 — Cloud Discovery flowing . . . 14
Test 3 — App governance and OAuth policy active . . . 14
Test 4 — Session/access policy scoped and report-only . . . 15
Test 5 — Conditional Access App Control routes sessions . . . 15
Test 6 — Cloud-app signal streaming . . . 15
Test 7 — A monitor policy fires (no enforcement) . . . 15
Test 8 — Enforcement/revoke stays approval-gated . . . 15
6. Troubleshooting Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps 16
Connector not connecting . . . 16
No discovery data . . . 16
App governance not inventorying . . . 16
App Control not proxying . . . 16
No cloud-app signal . . . 16
Policy too noisy / false positives . . . 17
Policy enforced or app revoked unexpectedly . . . 17
Script cannot authenticate . . . 17
7. Best Practices — Onboarding Accelerator: Microsoft Defender for Cloud Apps 17
Start every policy in report-only, scoped to a pilot . . . 17
Connect Microsoft 365 first, then priority third-party apps . . . 17
Use the Defender for Endpoint feed for discovery where you can . . . 17
Treat OAuth app governance as a standing risk surface . . . 18
Keep enforcing and revoke actions approval-gated until trusted . . . 18
Let discovery and hunting tables warm up . . . 18
Keep licensing and preview facts current . . . 18
Defer the data-governance decisions to the owners . . . 18
Where to go next . . . 18
8. Deliverables Summary — Onboarding Accelerator: Microsoft Defender for Cloud Apps 19

Documentation 19
PowerShell scripts (Microsoft Graph Security / Entra + KQL) 19
What the buyer receives 20
Outcome 20
Appendix A — PowerShell Scripts 21

Executive Summary — Onboarding Accelerator: Microsoft Defender for Cloud Apps

White-label · Defender

Most tenants that own Microsoft Defender for Cloud Apps (MDCA) run it as an empty console. The **app connectors** were never added, so the SaaS apps that hold the organization's data (Microsoft 365, and third-party apps like Salesforce, ServiceNow, Google Workspace, AWS) send no signal; **Cloud Discovery** was never wired to a log source, so shadow IT is invisible; **app governance** and **OAuth app** policies are off, so risky third-party app consents go unwatched; and **session/access policies** via Conditional Access App Control were never created, so there is no real-time control over risky sessions. The result is a **cloud access security broker (CASB)** that is licensed but blind. This accelerator onboards MDCA **end to end**: it connects the API app connectors, turns on Cloud Discovery, enables app governance and OAuth app policies, and configures session and access policies via **Conditional Access App Control** — piloted first on a small user group, validated, then expanded.

This is a complete, **deploy-it-today** package written for an administrator with no prior MDCA experience. It onboards a **pilot user group first**, keeps every enforcing policy in a **monitor/report-only** posture until validated, and never blocks production access to "test."

What you get

- A plain-language **Solution Overview** of app connectors, Cloud Discovery, app governance, OAuth app policies, and session/access control.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite.
- An **Implementation Guide** of exact, ordered steps — connect apps, turn on discovery, enable app governance, create monitor-first policies, wire Conditional Access App Control — pilot-first.
- Five **PowerShell scripts** (Microsoft Graph Security + Entra) plus **KQL advanced-hunting** queries for readiness discovery, connector/discovery/policy validation, coverage & OAuth-risk reporting, a `-WhatIf`-gated policy-scoping helper, and a `CloudAppEvents` validation hunt.
- **Testing** and **Troubleshooting** guides for a confident rollout.

What this delivers

A validated, monitored Defender for Cloud Apps deployment delivering **cloud app security / CASB** — connectors streaming from priority SaaS apps, Cloud Discovery surfacing shadow IT, app governance and OAuth policies watching risky consents, and session/access policies enforcing real-time control on a pilot scope — onboarded in a ring and expanded with confidence, not a half-configured console.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — connect apps and discovery, scope monitor-first policies to a pilot user group, validate signal and control, then expand tenant-wide.

Boundary

This solution **onboards the Defender for Cloud Apps pillar itself**. It is distinct from the unified Defender XDR onboarding accelerator and from the data-loss-prevention solution that governs cloud-app DLP — cross-reference those, do not duplicate them. Data-classification, acceptable-use, and incident-response determinations remain the customer's compliance, security, and operations functions; this guide provides the technical configuration.

Effort

A competent administrator can complete the pilot-scope connector, discovery, and policy validation in a focused engagement, then expand tenant-wide on a controlled cadence. Connector rollout scales with the number of SaaS apps to govern.

1. Solution Overview — Onboarding Accelerator: Microsoft Defender for Cloud Apps

What Microsoft Defender for Cloud Apps is

Microsoft Defender for Cloud Apps (MDCA) is Microsoft's **cloud access security broker (CASB)** and the SaaS-security pillar of Microsoft Defender. It gives four capabilities that together provide **cloud app security**:

- **App connectors (API-based)** — deep, out-of-band integrations with major SaaS apps (Microsoft 365, and third-party apps such as Salesforce, ServiceNow, Google Workspace, AWS, Box) that pull activity logs, files, accounts, and settings so MDCA can detect threats and apply file/governance policies.
- **Cloud Discovery** — analysis of traffic logs (from Defender for Endpoint's built-in feed, or a log collector / secure web gateway) to surface **shadow IT**: which cloud apps are in use, by whom, and at what risk.
- **App governance and OAuth app policies** — visibility and policy over the **third-party OAuth apps** users consent to in the tenant, catching over-privileged or malicious app grants.
- **Session and access policies (via Conditional Access App Control)** — real-time, in-line control of user sessions (block download, restrict copy/paste, require read-only) proxied through MDCA and triggered by Entra Conditional Access.

Most administration is in the unified Defender portal at `security.microsoft.com` under **Cloud apps**; Conditional Access App Control is wired in Entra.

Why this matters

Organizational data has moved into SaaS, but the security controls often have not. Without **connectors**, MDCA cannot see what happens inside the apps that hold the data. Without **Cloud Discovery**, shadow IT — unsanctioned apps staff adopt on their own — stays invisible until it causes an incident. Without **app governance / OAuth policies**, a single malicious or over-permissioned OAuth grant can exfiltrate mail and files with no password ever stolen. Without **session/access policies**, there is no way to allow a risky sign-in but constrain what it can do. This accelerator turns the empty CASB into a working one — connected, discovering, governing OAuth apps, and controlling sessions — validated on a pilot scope

before it can affect production access.

How it works

1. **App connectors** are added for the priority SaaS apps under **Cloud apps** → **Connected apps** → **App connectors**; each connector pulls activity, files, and accounts via the app's API (out of band — no user impact).

2. **Cloud Discovery** is enabled — the simplest path is the built-in **Defender for Endpoint integration** so onboarded devices feed discovery automatically; a **log collector** or SWG feed is used where MDE is not the source.

3. **App governance** is turned on and **OAuth app policies** are created to flag/limit risky third-party app consents (over-privileged scopes, unverified publishers, unusual consent patterns).

4. **Policies** (activity, file, anomaly-detection, session, access) start **monitor / report-only**, scoped to a **pilot user group**, so behavior is observed before anything enforces.

5. **Session and access policies** are wired through **Conditional Access App Control** in Entra — a CA policy routes the pilot users' sessions to MDCA, which then applies the session controls (e.g. block download on unmanaged devices).

6. **Validation** confirms `CloudAppEvents` and discovery data stream and that a scoped monitor policy generates the expected alert.

7. Enforcing controls are only broadened from the pilot after they are validated.

What you can control

• **Which apps are connected** and the order they onboard (Microsoft 365 first, then priority third-party apps).

• **Cloud Discovery source** — MDE integration, log collector, or SWG feed.

• **OAuth app governance** — which app consents are flagged, limited, or revoked (revocation approval-gated).

• **Policy posture** — monitor/report-only first, scoped to a pilot user group, before enforcing.

• **Session/access controls** — what a proxied session may do (download, copy, print, read-only) and for whom.

Limitations (be honest with buyers)

• MDCA and app governance are **license-gated** (Defender for Cloud Apps; app governance may require a specific plan). Onboarding without the plan fails or limits features — *verify current availability in your tenant.*

• **Session/access policies require Conditional Access App Control**, which depends on Entra ID P1/P2 Conditional Access and reverse-proxy session routing — *verify current licensing and supported apps.*

• **Cloud Discovery** depends on a log source; without the MDE feed or a log collector there is no shadow-IT visibility.

• Discovery, connectors, and hunting tables **populate over time** — allow warm-up after onboarding.

• MDCA controls cloud-app access and data movement; it does not replace endpoint EDR, email protection, identity hardening, or the customer's DLP-classification and acceptable-use decisions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com` → **Cloud apps**) for connectors, discovery, app governance, OAuth apps, and policies.

• **Entra** (`entra.microsoft.com`) for **Conditional Access App Control** that routes sessions to MDCA.

• **Microsoft Graph Security** and the **Microsoft 365 Defender** APIs (via the `Microsoft.Graph` PowerShell modules) for read-only discovery, validation, reporting, and advanced-hunting (`CloudAppEvents`, `OAuthAppInfo`, `AlertInfo`).

• Least-privilege roles: **Security Administrator** for configuration, **Security Reader** for read-only work, and **Global Administrator** only for one-time first-service activation and initial OAuth consent.

The standard deployment model

> ***Assess readiness (license, roles, apps, discovery source) → connect apps and enable discovery/app-governance → scope monitor-first policies to a pilot user group and wire Conditional Access App Control → validate signal and control on the pilot → expand tenant-wide.*** *Repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, with enforcing/revoke actions kept approval-gated until trusted.*

2. Requirements Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to onboarding Defender for Cloud Apps**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Defender for Cloud Apps activated

Description: MDCA initialized for the tenant under `security.microsoft.com` → **Cloud apps**. **Why:** Connectors, discovery, app governance, and policies only appear after first-time activation. **Verify:** Sign in as a Security Administrator and confirm **Cloud apps** → **Cloud discovery / Connected apps / Policies** load. Run `MCAONB-Discovery.ps1`. **Implement-if-missing:** Complete the one-time MDCA activation (Global Administrator required for first-service activation only).

2.2 Priority SaaS apps and connector credentials

Description: The list of SaaS apps to connect first (Microsoft 365, then priority third-party apps) and the admin credentials/consent each connector needs. **Why:** App connectors are API integrations; each requires an admin consent or API credential for that app. **Verify:** A prioritized app list exists; `MCAONB-Discovery.ps1` reports which connectors are already added. **Implement-if-missing:** Add the connector for the top app(s) under **Cloud apps** → **Connected apps** → **App connectors** and complete each app's consent flow. Name the effort `MDCA App Connectors – Priority SaaS – Prod`.

2.3 Cloud Discovery log source

Description: A traffic-log source for Cloud Discovery — the built-in **Defender for Endpoint integration**, a **log collector**, or a secure web gateway feed. **Why:** Without a log source there is no shadow-IT discovery; the MDE feed is the lowest-friction option if endpoints are onboarded to MDE. **Verify:** `security.microsoft.com` → **Cloud apps** → **Cloud discovery** → **Settings**; confirm a source is configured. `MCAONB-Discovery.ps1` reports the discovery source. **Implement-if-missing:** Turn on the **Defender for Endpoint** → **Cloud Apps** integration (if devices are MDE-onboarded — see the adjacent MDE onboarding accelerator), or deploy a log collector for a SWG.

2.4 App governance enabled

Description: App governance (OAuth app monitoring and policy) turned on for the tenant. **Why:** Risky third-party OAuth app consents are a major exfiltration path; app governance is what watches them. **Verify:** `security.microsoft.com` → **Cloud apps** → **App governance**; confirm it is enabled. `MCAONB-Discovery.ps1` reports app-governance status. *App governance may require a specific plan — verify current availability.* **Implement-if-missing:** Enable app governance and allow it to inventory the tenant's OAuth apps before creating policies.

2.5 Entra Conditional Access for App Control

Description: Entra Conditional Access (P1/P2) available to route pilot users' sessions to MDCA for **Conditional Access App Control**. **Why:** Session and access policies are enforced by proxying sessions, triggered by a Conditional Access policy. **Verify:** `entra.microsoft.com` → **Protection** → **Conditional Access**; confirm CA is available and a policy can target the pilot users. *Verify current App Control licensing and supported apps.* **Implement-if-missing:** Confirm Entra ID P1/P2, then plan the CA policy `MDCA Session Control – Pilot Users – Report` (Step 5).

2.6 Pilot user group defined

Description: A named pilot user group `MDCA Pilot Users – Pilot Ring – Pilot` to scope monitor-first policies and session control. **Why:** Policies (especially session/access) are validated on a pilot group before tenant-wide enforcement (per `global-requirements-defender.md` G6). **Verify:** The group exists in Entra; `MCAONB-Validate.ps1 -Check Policy` checks the scope. **Implement-if-missing:** Create the group `MDCA Pilot Users – Pilot Ring – Pilot` before scoping any policy.

2.7 Policy posture and enforcement governance

Description: Agreement to start every policy in **monitor / report-only** and keep enforcing actions (block, revoke OAuth app, suspend session) approval-gated during the pilot. **Why:** Enforcing policies can disrupt

legitimate work; the organization decides the guardrails (per `global-requirements-defender.md` G8). **Verify:** A documented policy posture exists; policies are report-only first. `MCAONB-Validate.ps1 -Check Policy` reports the posture. **Implement-if-missing:** Create policies in report-only mode; *enforcement and OAuth-revocation decisions are the security team's — this guide covers the technical configuration.*

Checklist

- MDCA activated — `MCAONB-Discovery.ps1`
- Priority SaaS apps + connector credentials (`MDCA App Connectors – Priority SaaS – Prod`)
- Cloud Discovery log source (MDE integration / log collector / SWG)
- App governance enabled
- Entra Conditional Access available for App Control
- Pilot user group `MDCA Pilot Users – Pilot Ring – Pilot` defined
- Policy posture (report-only first, enforcement approval-gated) documented

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps

Follow these steps **in order**. Most configuration is in the **unified Defender portal** (`https://security.microsoft.com` → **Cloud apps**); Conditional Access App Control is wired in the **Entra admin center** (`https://entra.microsoft.com`). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Connect the apps and discovery, scope every policy **monitor / report-only** to a **pilot user group** first, validate that signal and control work, and only then enforce and expand. Keep enforcing actions (block, revoke OAuth app, suspend session) approval-gated throughout. Never block production access to "test."*

Naming (set once, reuse verbatim)

- Pilot user group (Entra): `MDCA Pilot Users – Pilot Ring – Pilot`
- App connectors effort: `MDCA App Connectors – Priority SaaS – Prod`
- OAuth app policy: `MDCA OAuth App Risk – Tenant – Report`
- Session policy (via CA App Control): `MDCA Session Control – Pilot Users – Report`
- Access policy: `MDCA Access Control – Pilot Users – Report`
- Conditional Access policy (Entra): `MDCA Session Control – Pilot Users – Report`
- Advanced-hunting saved query: `MDCA Signal Validation – Tenant – Validate`

Step 1 — Assess readiness

1. Sign in to `security.microsoft.com` as a **Security Administrator**.
2. Run `MCAONB-Discovery.ps1` to capture the current state: MDCA activation, plan hints, admin roles, existing app connectors, Cloud Discovery source, and app-governance status.
3. In **Entra** → **Groups**, create the pilot user group `MDCA Pilot Users – Pilot Ring – Pilot`.

Step 2 — Connect the priority apps (API connectors)

1. `security.microsoft.com` → **Cloud apps** → **Connected apps** → **App connectors** → **Connect an app**.
2. Connect **Microsoft 365** first, then the priority third-party apps as `MDCA App Connectors – Priority SaaS – Prod`; complete each app's admin consent / API-credential flow.
3. Confirm each connector shows **Connected** and begins pulling activity (connectors work out of band — no user impact).
4. Run `MCAONB-Validate.ps1 -Check Connectors` to confirm the connectors are connected and streaming.

Step 3 — Turn on Cloud Discovery

1. `security.microsoft.com` → **Cloud apps** → **Cloud discovery** → **Settings**.
2. Enable the **Microsoft Defender for Endpoint integration** so MDE-onboarded devices feed discovery automatically (preferred). Where MDE is not the source, configure a **log collector** for the

secure web gateway.

3. Confirm discovery data appears under **Cloud apps** → **Cloud discovery** → **Dashboard**.

4. Run `MCAONB-Validate.ps1 -Check Discovery` to confirm discovery data is flowing.

Step 4 — Enable app governance and OAuth app policy (report-only)

1. `security.microsoft.com` → **Cloud apps** → **App governance** — enable it and let it inventory the tenant's OAuth apps. *App governance may require a specific plan — verify current availability.*

2. Create the OAuth app policy `MDCA OAuth App Risk – Tenant – Report` in **report-only** to flag over-privileged scopes, unverified publishers, and unusual consent patterns — **no auto-revoke** yet.

3. Run `MCAONB-Report.ps1 -Section OAuth` to list the current risky OAuth apps.

4. *Revoking an app consent is disruptive and stays approval-gated — the decision is the security team's.*

Step 5 — Create session/access policies via Conditional Access App Control (pilot, report-only)

1. In **Entra** → **Protection** → **Conditional Access** → **New policy**, create `MDCA Session Control – Pilot Users – Report`: assign **Users =** `MDCA Pilot Users – Pilot Ring – Pilot`, target cloud apps, and under **Session** choose **Use Conditional Access App Control** → **Use custom policy** (routes sessions to MDCA). *Verify current App Control licensing and supported apps.*

2. In `security.microsoft.com` → **Cloud apps** → **Policies** → **Policy management**, create the session policy `MDCA Session Control – Pilot Users – Report` (e.g. monitor downloads on unmanaged devices) in **monitor** mode, and the access policy `MDCA Access Control – Pilot Users – Report` in **monitor** mode — both scoped to the pilot users.

3. Run `MCAONB-Validate.ps1 -Check Policy` to confirm the policies exist, are report-only, and are scoped to the pilot group.

Step 6 — Add anomaly-detection and activity policies (monitor-first)

1. `security.microsoft.com` → **Cloud apps** → **Policies** — confirm the built-in **anomaly detection** policies are on (impossible travel, mass download, risky OAuth), tuned to the pilot scope where possible.

2. Add any needed **activity** or **file** policies in **report-only** first.

3. Run `MCAONB-Report.ps1 -Section Policies` to capture the current policy inventory and posture.

Step 7 — Validate signal and control on the pilot

1. Run `MCAONB-Hunt.ps1` to execute `MDCA Signal Validation – Tenant – Validate` — it confirms `CloudAppEvents` (and OAuth app data) stream from the connectors and discovery.

2. As a pilot user, perform a benign action the monitor policy watches (e.g. a download from a connected app on an unmanaged device) and confirm the expected **alert** appears in **Incidents &**

alerts — without enforcement blocking anything.

3. Complete the **Testing Guide** before enforcing or expanding.

Step 8 — Enforce and expand tenant-wide

1. **Policies:** after validation, move the pilot session/access policies from **monitor** to **enforce** for the pilot group, confirm no false positives, then broaden the CA policy `MDCA Session Control – Pilot Users – Report` scope in waves toward the full tenant.

2. **Connectors:** add connectors for the remaining SaaS apps.

3. **OAuth governance:** review flagged apps with the security team; only then consider approval-gated revocation for the riskiest.

4. **Discovery:** expand the discovery source coverage (more MDE devices / log-collector sources).

5. Re-baseline with `MCAONB-Report.ps1 -SaveBaseline` and track coverage/OAuth-risk drift each cycle.

Order of operations

`Assess → connect apps (M365 first) → turn on Cloud Discovery → enable app governance + OAuth report policy → create session/access policies via CA App Control (pilot, report-only) → add anomaly/activity policies → validate signal + control → enforce + expand`

5. Testing Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps

Run these checks after the pilot onboarding (Steps 2–7) and again after each expansion wave. Each test has a **pass/fail gate**; do not enforce or expand until the pilot passes. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — App connectors connected

Procedure: Run `MCAONB-Validate.ps1 -Check Connectors`. In `security.microsoft.com` → **Cloud apps** → **Connected apps** → **App connectors**, confirm each connector in `MDCA App Connectors – Priority SaaS – Prod` shows **Connected**. **Pass:** Microsoft 365 (and each priority app) is connected and pulling activity. **Fail:** A connector shows an error/disconnected — see Troubleshooting *Connector not connecting*.

Test 2 — Cloud Discovery flowing

Procedure: Run `MCAONB-Validate.ps1 -Check Discovery`. In **Cloud apps** → **Cloud discovery** → **Dashboard**, confirm discovered apps and traffic appear. **Pass:** Discovery data (apps, users, traffic) is visible from the configured source. **Fail:** Dashboard empty — see Troubleshooting *No discovery data*.

Test 3 — App governance and OAuth policy active

Procedure: Run `MCAONB-Report.ps1 -Section OAuth`. In **Cloud apps** → **App governance**, confirm OAuth apps are inventoried and `MDCA OAuth App Risk – Tenant – Report` is present and report-only. **Pass:** App governance is on, OAuth apps are inventoried, and the report policy flags risky apps without revoking. **Fail:** App governance off or no inventory — see Troubleshooting *App governance not inventorying*.

Test 4 — Session/access policy scoped and report-only

Procedure: Run `MCAONB-Validate.ps1 -Check Policy`. In **Cloud apps** → **Policies**, confirm `MDCA Session Control – Pilot Users – Report` and `MDCA Access Control – Pilot Users – Report` exist, are **monitor/report-only**, and are scoped to `MDCA Pilot Users – Pilot Ring – Pilot`. **Pass:** Both policies exist, are report-only, and target only the pilot users. **Fail:** Policy missing, enforcing, or wrongly scoped — re-create per Implementation Step 5.

Test 5 — Conditional Access App Control routes sessions

Procedure: As a pilot user, sign in to a targeted app; confirm the session is proxied (the MDCA session banner/URL suffix appears) per the CA policy `MDCA Session Control – Pilot Users – Report`. **Pass:** The pilot user's session is routed through MDCA (App Control active) without blocking access. **Fail:** Session not proxied — see Troubleshooting *App Control not proxying*.

Test 6 — Cloud-app signal streaming

Procedure: Run `MCAONB-Hunt.ps1` to execute `MDCA Signal Validation – Tenant – Validate`. Confirm `CloudAppEvents` returns recent rows from the connectors/discovery. **Pass:** `CloudAppEvents` streams recent activity. **Fail:** Empty — see Troubleshooting *No cloud-app signal*.

Test 7 — A monitor policy fires (no enforcement)

Procedure: As a pilot user, perform a benign action the monitor policy watches (e.g. a download on an unmanaged device). In **Incidents & alerts**, confirm the expected alert appears — and access is **not** blocked. **Pass:** The monitor policy generates the alert; the pilot user is not disrupted. **Fail:** No alert or access was blocked — re-check the policy mode/scope (Implementation Step 5).

Test 8 — Enforcement/revoke stays approval-gated

Procedure: Run `MCAONB-Report.ps1 -Section Policies` and confirm no policy auto-blocks or auto-revokes during the pilot. **Pass:** All enforcing/revoke actions require approval; nothing auto-executes on the pilot. **Fail:** An enforcing action is live before agreed — reset to report-only and confirm with the security team.

Exit gate

All eight tests pass on the pilot user group **before** you enforce any policy or expand the scope (Implementation Step 8). Re-run Tests 1, 2, 6, and 7 after each expansion wave and compare against the saved baseline.

6. Troubleshooting Guide — Onboarding Accelerator: Microsoft Defender for Cloud Apps

Common field issues, in **symptom** → **cause** → **fix** form. Run `MCAONB-Discovery.ps1` and `MCAONB-Validate.ps1` first — most causes surface there.

Connector not connecting

Symptom: Test 1 fails — a connector in `MDCA App Connectors – Priority SaaS – Prod` shows an error or "Disconnected." **Cause:** The app's admin consent/API credential was not completed or has expired, the connecting account lacks admin rights in that app, or the app's API access is restricted. **Fix:** Re-run the connector's consent flow with an admin of that app; confirm the API credential/token is valid; check the app's own API access settings. Remove and re-add the connector if needed, then re-run `MCAONB-Validate.ps1 -Check Connectors`.

No discovery data

Symptom: Test 2 fails — **Cloud discovery** → **Dashboard** is empty. **Cause:** No discovery source is configured, the Defender for Endpoint integration is off, pilot devices are not MDE-onboarded, or the log collector is not receiving logs. **Fix:** Confirm a source under **Cloud discovery** → **Settings**; turn on the **Defender for Endpoint** → **Cloud Apps** integration and confirm devices are MDE-onboarded (see the MDE onboarding accelerator); verify the log collector is receiving SWG logs. Allow warm-up, then re-run `MCAONB-Validate.ps1 -Check Discovery`.

App governance not inventorying

Symptom: Test 3 fails — **App governance** shows no OAuth app inventory, or the report policy is absent. **Cause:** App governance is not enabled, the plan does not include it, or it has not finished the initial inventory. **Fix:** Enable app governance under **Cloud apps** → **App governance**; *verify the plan includes app governance in your tenant*; allow the initial inventory to complete, then create `MDCA OAuth App Risk – Tenant – Report`. Re-run `MCAONB-Report.ps1 -Section OAuth`.

App Control not proxying

Symptom: Test 5 fails — a pilot user's session is not routed through MDCA (no session banner/suffix). **Cause:** The Conditional Access policy `MDCA Session Control – Pilot Users – Report` is not enabled/assigned to the pilot users, the app is not supported by App Control, or the session-control session setting is not applied. **Fix:** Confirm the CA policy is **On** and assigned to `MDCA Pilot Users – Pilot Ring – Pilot`; confirm the app is supported by Conditional Access App Control (*verify current supported apps*); confirm **Session** → **Use Conditional Access App Control** is set. Have the user sign out/in and re-test.

No cloud-app signal

Symptom: Test 6 fails — `CloudAppEvents` returns no rows. **Cause:** Connectors are newly added and still warming up, or discovery has no source, so no activity is streaming yet. **Fix:** Confirm connectors are connected (Test 1) and discovery is flowing (Test 2); allow warm-up (tables populate over hours); re-run

`MCAONB-Hunt.ps1`.

Policy too noisy / false positives

Symptom: A monitor policy generates excessive or irrelevant alerts on the pilot. **Cause:** The policy scope or thresholds are too broad, or a legitimate business workflow matches the pattern. **Fix:** Tighten the policy scope to `MDCA Pilot Users – Pilot Ring – Pilot`, adjust thresholds/filters, and add exclusions for known-good workflows. Keep it report-only until the noise is acceptable, then enforce.

Policy enforced or app revoked unexpectedly

Symptom: A pilot user was blocked, or an OAuth app was revoked, during the pilot. **Cause:** A policy was created in enforce mode, or an app-governance policy had auto-revoke enabled, before validation. **Fix:** Reset the policy to **report-only** (Implementation Step 5) and disable auto-revoke; confirm with the security team; re-validate before enforcing. *Enforcement/revocation decisions are the security team's.*

Script cannot authenticate

Symptom: `MCAONB-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, consent to the required read scopes was not granted, or the account lacks Security Reader. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm Security Reader (or higher). Re-run the script.

7. Best Practices — Onboarding Accelerator: Microsoft Defender for Cloud Apps

Field-proven guidance for keeping the Defender for Cloud Apps deployment clean, useful, and low-risk.

Start every policy in report-only, scoped to a pilot

- Create activity, file, session, and access policies in **monitor / report-only** first, scoped to `MDCA Pilot Users – Pilot Ring – Pilot`. Watch behavior, tune out false positives, then enforce. A CASB that blocks legitimate work on day one loses the room.
- Move to enforce for the pilot, confirm, then broaden the Conditional Access scope in waves — never enforce tenant-wide first.

Connect Microsoft 365 first, then priority third-party apps

- The Microsoft 365 connector gives the fastest, richest signal. Add high-value third-party apps (`MDCA App Connectors – Priority SaaS – Prod`) next by data sensitivity, not by ease. Connectors are out-of-band API integrations — they add signal without user impact.

Use the Defender for Endpoint feed for discovery where you can

- If devices are MDE-onboarded, the built-in **Defender for Endpoint** → **Cloud Apps** integration is the lowest-friction Cloud Discovery source — no log collector to maintain. Fall back to a log collector / SWG

feed only where MDE is not the source.

Treat OAuth app governance as a standing risk surface

• Risky **OAuth app** consents exfiltrate data with no stolen password. Keep `MDCA OAuth App Risk – Tenant – Report` running, review flagged apps (over-privileged scopes, unverified publishers) on a cadence, and keep **revocation approval-gated** — it can break legitimate integrations. Revocation decisions belong to the security team.

Keep enforcing and revoke actions approval-gated until trusted

• Block, suspend-session, and revoke-app actions are disruptive. Leave them approval-gated during the pilot and early expansion; moving toward automatic enforcement is the security team's decision, made after validation.

Let discovery and hunting tables warm up

• Discovery data and `CloudAppEvents` populate over time. Use `MCAONB-Hunt.ps1` to confirm streaming before judging coverage, and re-baseline with `MCAONB-Report.ps1 -SaveBaseline` after each expansion.

Keep licensing and preview facts current

• MDCA, **app governance**, and **Conditional Access App Control** are license-gated and evolve, and App Control supports a changing set of apps. Before relying on a capability, **verify current availability, licensing, and supported apps in your tenant.**

Defer the data-governance decisions to the owners

• What counts as sensitive data, which apps are sanctioned, and acceptable-use policy are **the customer's compliance/security determinations**. This solution delivers the technical CASB onboarding and controls; the classification and policy decisions stay with the owners.

Where to go next

Once MDCA is onboarded and validated, natural next steps in the catalog:

• The **Cloud App DLP with Defender for Cloud Apps** solution to layer data-loss-prevention on the connected apps.

• The **Microsoft Defender XDR — Unified Onboarding** accelerator to correlate MDCA with Endpoint, Identity, and Office 365 in one portal.

• The **Onboarding Accelerator — Microsoft Defender for Endpoint** to establish the MDE feed that powers Cloud Discovery.

• **Entra Conditional Access baseline** and identity-governance solutions to strengthen the access layer that App Control depends on.

8. Deliverables Summary — Onboarding Accelerator: Microsoft Defender for Cloud Apps

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What MDCA is — app connectors, Cloud Discovery, app governance/OAuth policies, session/access control; benefits, limitations
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — connect apps, discovery, app governance, monitor-first policies, Conditional Access App Control, validate, enforce/expand
5. Testing Guide	Eight validation tests with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations and where to go next

PowerShell scripts (Microsoft Graph Security / Entra + KQL)

Script	What it does		
`MCAONB-Discovery.ps1`	Read-only readiness — MDCA activation, plan hints, admin roles, existing app connectors, Cloud Discovery source, app-governance status		
`MCAONB-Validate.ps1`	Targeted validation (`-Check Connectors\	Discovery\	Policy`) — confirms connectors, discovery flow, and that session/access policies are report-only and pilot-scoped
`MCAONB-Report.ps1`	Coverage report by section (`-Section OAuth\	Policies\	All`) — risky OAuth apps, policy inventory/posture, and drift versus a saved baseline

Script	What it does		
`MCAONB-ScopePolicy.ps1`	`-WhatIf`-gated helper that scopes/updates the Conditional Access App Control policy to the pilot user group (safe preview by default)		
`MCAONB-Hunt.ps1`	Runs the KQL advanced-hunting query `MDCA Signal Validation` via Graph Security to confirm `CloudAppEvents`/OAuth data stream (read-only)		

What the buyer receives

- This solution as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-scoped, monitor-first onboarding with enforcing/revoke actions kept approval-gated until trusted.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A validated, monitored Defender for Cloud Apps deployment delivering CASB — connectors streaming from priority SaaS apps, Cloud Discovery surfacing shadow IT, app governance and OAuth policies watching risky consents, and session/access policies enforcing real-time control on a pilot scope — proven before you enforce tenant-wide.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MCAONB-Discovery.ps1

Purpose: Read-only discovery for the "Onboarding Accelerator — Microsoft Defender for Cloud Apps" solution. Reports the signed-in context, licensed MDCA plan hints, admin roles, existing app connectors, the Cloud Discovery source, app-governance status, and whether Conditional Access is available for App Control. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MCAONB-Discovery.ps1 Requires the Microsoft.Graph modules (Microsoft.Graph.Authentication and Identity.SignIns for Conditional Access read). Connects read-only with Connect-MgGraph. EXPECTED RESULTS A readiness picture: tenant/context, MDCA plan hints, admin roles, connectors already added, discovery source, app-governance status, and CA availability — the state to confirm before onboarding. Written to a JSON snapshot. .NOTES White-label. Read-only. Some MDCA settings surface through the portal only — the script prints the exact portal path where an API read is unavailable. Licensing facts are indicative — verify. > [CmdletBinding()] param([string]$OutputPath = ".\MCAONB-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MCAONB-Discovery.ps1 Requires the Microsoft.Graph modules (Microsoft.Graph.Authentication and Identity.SignIns for Conditional Access read). Connects read-only with Connect-MgGraph. EXPECTED RESULTS A readiness picture: tenant/context, MDCA plan hints, admin roles, connectors already added, discovery source, app-governance status, and CA availability — the state to confirm before onboarding. Written to a JSON snapshot. .NOTES White-label. Read-only. Some MDCA settings surface through the portal only — the script prints the exact portal path where an API read is unavailable. Licensing facts are indicative — verify. > [CmdletBinding()] param([string]$OutputPath = ".\MCAONB-discovery.json")

MCAONB-Hunt.ps1

Purpose: Read-only advanced-hunting for the "Onboarding Accelerator — Microsoft Defender for Cloud Apps" solution. Executes the saved query 'MDCA Signal Validation - Tenant - Validate' via Microsoft Graph Security advanced hunting to confirm CloudAppEvents are streaming from the connectors/discovery and to surface recent activity for a pilot user so you can confirm a monitor policy fired (Testing Guide Tests 6 and 7). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MCAONB-Hunt.ps1 -PilotUserUpn 'pilot.user@contoso.com' Uses Microsoft.Graph.Authentication and the ThreatHunting.Read.All scope. Read-only. EXPECTED RESULTS Row counts for CloudAppEvents and recent activity for the pilot user — proof the CASB is streaming and a monitor policy generated signal without enforcement. .NOTES White-label. Read-only (advanced hunting). Embedded KQL below is the delivered query. > [CmdletBinding()] param([string]$PilotUserUpn = 'REPLACE-with-pilot-user-upn@contoso.com', [int]$LookbackDays = 1)

How to run: powershell -ExecutionPolicy Bypass -File .\MCAONB-Hunt.ps1 -PilotUserUpn 'pilot.user@contoso.com' Uses Microsoft.Graph.Authentication and the ThreatHunting.Read.All scope. Read-only. EXPECTED RESULTS Row counts for CloudAppEvents and recent activity for the pilot user — proof the CASB is streaming and a monitor policy generated signal without enforcement. .NOTES White-label. Read-only (advanced hunting). Embedded KQL below is the delivered query. > [CmdletBinding()] param([string]$PilotUserUpn = 'REPLACE-with-pilot-user-upn@contoso.com', [int]$LookbackDays = 1)

MCAONB-Report.ps1

Purpose: Read-only reporting for the "Onboarding Accelerator — Microsoft Defender for Cloud Apps" solution. Summarizes risky third-party OAuth apps (via advanced hunting / app governance signal), the policy inventory and posture (should be report-only during the pilot), and — with -SaveBaseline — records a baseline so later runs show drift. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MCAONB-Report.ps1 -Section All -SaveBaseline -Section accepts: OAuth | Policies | All (default All). Uses Microsoft.Graph.Authentication + Graph Security (advanced hunting, secure score). EXPECTED RESULTS A list of risky OAuth apps, a policy posture summary, and drift versus the saved baseline. A downward trend in risky-OAuth count / a stable report-only posture = healthy onboarding. .NOTES White-label. Read-only. Revoking an OAuth app is disruptive and stays approval-gated — the security team decides. OAuth-risk data is indicative; verify in App governance. > [CmdletBinding()] param([ValidateSet('OAuth','Policies','All')] [string]$Section = 'All', [switch]$SaveBaseline, [string]$BaselinePath = ".\MCAONB-baseline.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MCAONB-Report.ps1 -Section All -SaveBaseline -Section accepts: OAuth | Policies | All (default All). Uses Microsoft.Graph.Authentication + Graph Security (advanced hunting, secure score). EXPECTED RESULTS A list of risky OAuth apps, a policy posture summary, and drift versus the saved baseline. A downward trend in risky-OAuth count / a stable report-only posture = healthy onboarding. .NOTES White-label. Read-only. Revoking an OAuth app is disruptive and stays approval-gated — the security team decides. OAuth-risk data is indicative; verify in App governance. > [CmdletBinding()] param([ValidateSet('OAuth','Policies','All')] [string]$Section = 'All', [switch]$SaveBaseline, [string]$BaselinePath = ".\MCAONB-baseline.json")

MCAONB-ScopePolicy.ps1

Purpose: Change helper for the "Onboarding Accelerator — Microsoft Defender for Cloud Apps" solution. Ensures the Conditional Access policy 'MDCA Session Control - Pilot Users - Report' targets ONLY the pilot user group 'MDCA Pilot Users - Pilot Ring - Pilot' and is left in report-only (CA state = enabledForReportingButNotEnforced) during the pilot. This is the only script that can change configuration, and it defaults to a safe preview. HOW TO RUN Preview (default — shows what WOULD change, changes nothing): powershell -ExecutionPolicy Bypass -File .\MCAONB-ScopePolicy.ps1 `-PolicyName 'MDCA Session Control - Pilot Users - Report' -GroupName 'MDCA Pilot Users - Pilot Ring - Pilot' Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\MCAONB-ScopePolicy.ps1` -PolicyName 'MDCA Session Control - Pilot Users - Report' -GroupName 'MDCA Pilot Users - Pilot Ring - Pilot' -Apply Requires Microsoft.Graph with Policy.ReadWrite.ConditionalAccess and Group.Read.All. EXPECTED RESULTS Preview: prints the resolved policy id, group id, and the scope it WOULD set (pilot only, report-only). Apply: updates the policy scope and leaves it report-only. Validate with MCAONB-Validate.ps1 -Check Policy. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Keeps the policy report-only; enforcement is a separate, deliberate step after validation. Session controls are set in the MDCA portal. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PolicyName, [Parameter(Mandatory)] [string]$GroupName, [switch]$Apply)

How to run: Preview (default — shows what WOULD change, changes nothing): powershell -ExecutionPolicy Bypass -File .\MCAONB-ScopePolicy.ps1 `-PolicyName 'MDCA Session Control - Pilot Users - Report' -GroupName 'MDCA Pilot Users - Pilot Ring - Pilot' Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\MCAONB-ScopePolicy.ps1` -PolicyName 'MDCA Session Control - Pilot Users -

Report' -GroupName 'MDCA Pilot Users - Pilot Ring - Pilot' -Apply Requires Microsoft.Graph with Policy.ReadWrite.ConditionalAccess and Group.Read.All. EXPECTED RESULTS Preview: prints the resolved policy id, group id, and the scope it WOULD set (pilot only, report-only). Apply: updates the policy scope and leaves it report-only. Validate with MCAONB-Validate.ps1 -Check Policy. .NOTES White-label. DEFAULTS TO -WhatIf/preview. Keeps the policy report-only; enforcement is a separate, deliberate step after validation. Session controls are set in the MDCA portal. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$PolicyName, [Parameter(Mandatory)] [string]$GroupName, [switch]$Apply)

MCAONB-Validate.ps1

Purpose: Read-only validation for the "Onboarding Accelerator — Microsoft Defender for Cloud Apps" solution. Confirms the piece named by -Check: connectors connected, Cloud Discovery flowing, or the session/access policies report-only and scoped to the pilot group. Maps to the Testing Guide gates. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MCAONB-Validate.ps1 -Check Policy -Check accepts: Connectors | Discovery | Policy | All (default All). Uses Microsoft.Graph.Authentication and Graph Security. Reads only; makes no changes. EXPECTED RESULTS PASS/REVIEW lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before enforcing or expanding beyond the pilot. .NOTES White-label. Read-only. Where a setting is portal-only, the script prints the exact path to confirm manually. Policies must stay report-only until validated. > [CmdletBinding()] param([ValidateSet('Connectors','Discovery','Policy','All')] [string]$Check = 'All', [string]$PilotGroup = 'MDCA Pilot Users - Pilot Ring - Pilot')

How to run: powershell -ExecutionPolicy Bypass -File .\MCAONB-Validate.ps1 -Check Policy -Check accepts: Connectors | Discovery | Policy | All (default All). Uses Microsoft.Graph.Authentication and Graph Security. Reads only; makes no changes. EXPECTED RESULTS PASS/REVIEW lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before enforcing or expanding beyond the pilot. .NOTES White-label. Read-only. Where a setting is portal-only, the script prints the exact path to confirm manually. Policies must stay report-only until validated. > [CmdletBinding()] param([ValidateSet('Connectors','Discovery','Policy','All')] [string]$Check = 'All', [string]$PilotGroup = 'MDCA Pilot Users - Pilot Ring - Pilot')